

PotensicProxy_TAF

Projektdokumentation - Kamera-RX-/Download-Framing

Softwareversion 0.959 | Dokumentationsversion v1.6 | Stand 2026-10-03

Projektname	PotensicProxy_TAF
Dokumentenart	Projektdokumentation / Änderungsnachweis
Thema	Kamera-RX-Reassembly und Galerie-Download-Framing
Erstellungsdatum	2026-10-03
Verantwortlich	OpenAI / ChatGPT im Projekt Potensic_App
Dokumentversion	v1.6
Softwareversion	0.959 (Development / Pre-1.0)
Status	Implementiert, Geräte-Regressionsprüfung offen
Ablageort	DOCUMENTATION/ im PotensicProxy_TAF-Codepaket
Referenzen	Live-System-Log 2026-10-03 06:13:48; USB-XML-Log 2026-10-03 06:13:43; PotensicPro DownloadData/RemoteFileDownloader

1. Anlass und Fehlerbild

Beim Galerie-Download wurde der erste Nutzdatenblock mit 3072 Byte korrekt verarbeitet. Direkt danach verlor der bestehende Parser die Framegrenze. Im Live-System-Log erschienen Meldungen wie „Camera download stream lost framing“, unplausible Download-Offsets und scheinbare Framegrößen im fünfstelligen Bereich. Der bestätigte lokale Offset blieb bei 3072 Byte. Nach einem Passthrough-Abbruch wurde der Download folgerichtig mit „restart required after reconnect“ beendet.

Der Vergleich mit dem USB-Paketlog und der Original-App zeigte, dass die 0x1B-Downloadstruktur selbst nicht neu erfunden werden muss. PotensicPro verwendet weiterhin flag, optionalen 32-Byte-Abschlussbereich, uint64-LE-Offset, uint16-LE-Payloadlänge und Payload. Die kritische Schwachstelle lag eine Ebene davor: die Begrenzung der inneren FF-FE-Kameraframes innerhalb eines äußeren FE-0x05-Payloads.

2. Technische Ursache

Die bisherige TAF-Implementierung nahm bei FE 0x05 an, dass der gesamte äußere Payload genau einen inneren Kameraframe enthält. Die Command-Body-Grenze wurde deshalb bis unmittelbar vor das Ende des äußeren FE-Payloads gezogen. Enthielt ein FE-0x05-Payload mehrere innere FF-FE-Frames, wurden die Bytes des nachfolgenden Frames an den vorhergehenden Command-Body angehängt.

Für 0x1B war das besonders schädlich: Nach dem ersten vollständigen DownloadData-Datensatz begann der verbleibende Buffer mit dem FF-FE-Header des nächsten inneren Frames. Der Downloadparser interpretierte diesen Header anschließend als flag/offset/length eines weiteren Mediadatensatzes.

Dadurch entstanden zufällige große Offset- und Längenwerte. Dasselbe Grundproblem erklärte die zuvor beobachteten „0x39 trailing byte(s)“-Warnungen.

3. Umsetzung in Software 0.959

Persistenter innerer Kamera-RX-Buffer

FE-0x05-Payloads werden in einem eigenen Buffer gesammelt. Der Buffer bleibt über aufeinanderfolgende FE-Lieferungen erhalten.

Längengeführte FF-FE-Reassembly

Ein innerer RX-Frame beginnt mit FF FE. Die Little-Endian-Länge an Offset +2/+3 bestimmt die vollständige Framegröße als 4 + innerLength.

Mehrfachframes pro FE-Payload

Mehrere vollständige innere Frames in einem äußeren FE-Payload werden nacheinander und unabhängig verarbeitet.

Teilframes

Ein unvollständiger innerer Frame wird nicht verworfen und nicht im Payload neu gesucht, sondern bis zur nächsten FE-0x05-Lieferung gehalten.

Saubere Command-Grenze

0x39, 0x1B und alle anderen 0x0020-Kommandos erhalten ausschließlich die Bytes ihres eigenen vollständigen inneren Frames.

Downloadparser gehärtet

Der 0x1B-Body akzeptiert nur flag 0/1/2 und die deklarierte Payloadlänge. Byteweises Resynchronisieren mitten in Bild-/Videodaten wurde entfernt. Bei unmöglicher Body-Grenze wird der Download sauber abgebrochen.

Reconnect-Reset

Der neue innere Kamera-RX-Buffer wird beim Passthrough-Disconnect zusammen mit den transienten Kamera-/Downloadzuständen zurückgesetzt.

Startup-Reihenfolge

Kamera-Kommandos, die vor Passthrough-Bereitschaft ausgelöst werden, werden begrenzt zwischengespeichert und nach Reconnect/Status-Synchronisierung gesendet. Die bisherige Fehlermeldung „transport is not initialized“ wird damit vermieden.

Versionierung

Frontend und Android/Backend wurden auf 0.959 gesetzt; Android versionCode wurde auf 32 erhöht. Der Stand bleibt ausdrücklich Pre-1.0.

4. Protokollbezug

Die Änderung verändert keine bestätigten Kamera-Command-IDs. FE TX 0x15 / FE RX 0x05 und die innere Kamerafunktion 0x0020 bleiben unverändert. Ebenso bleiben die aus PotensicPro bestätigten 0x1B-Felder unverändert. Neu ist ausschließlich die korrekte, längengeführte Begrenzung und Reassembly der inneren FF-FE-Frames, bevor Command-Status und Command-Body ausgewertet werden.

Ebene	Kennung	Länge	Behandlung 0.959
Äußerer Transport	FE / Typ 0x05	FE-Header-Länge	UsbStreamDemuxer hält vollständige FE-Pakete
Innerer Kamera-RX	FF FE	uint16 LE @ +2/+3; total = 4 + len	persistentes Reassembly, mehrere Frames möglich
Kamerafunktion	0x0020 @ inner +4/+5	innerhalb des Frames	erst nach kompletter Reassembly auswerten
Download	Cmd 0x1B	flag/offset/u16 payloadLen	nur Bodybytes des eigenen inneren Frames zusammenführen

5. Validierung

Durchgeführt:

Vue/TypeScript-Prüfung: webui/node_modules/.bin/vue-tsc --noEmit - bestanden.

Synthetischer Reassembly-Test mit zwei realen FE-0x05/0x39-Innenframes aus dem USB-XML-Log: kombinierte und künstlich gesplittete Anlieferung ergab exakt die beiden ursprünglichen Frames - bestanden.

Bei demselben Test hätte die alte Body-Begrenzung 111 Byte des zweiten inneren Frames fälschlich dem ersten Command-Body zugeschlagen; die neue Begrenzung verhindert dies.

Der vollständige Vite-Produktionsbuild konnte in dieser Linux-Arbeitsumgebung nicht ausgeführt werden, weil das im gelieferten node_modules benötigte optionale Paket @rollup/rollup-linux-x64-gnu fehlt. Der Android-preBuild ist weiterhin so konfiguriert, dass er auf der Ziel-Buildumgebung den WebUI-Build ausführt und direkt nach app/src/main/assets/web schreibt.

6. Geräte-Regressionsprüfung für 0.959

Test	Aktion	Erwartung	Status
D1	Galerie öffnen und Listen laden	Foto-/Videoliste vollständig, keine Regression	offen

Test	Aktion	Erwartung	Status
D2	JPG > 3 KiB herunterladen	Fortschritt über 3072 Byte hinaus; keine lost- framing-/Random- Offset-Meldung	offen
D3	größere Datei / Video herunterladen	mehrere 0x1B- Einheiten bis 100 %, Android-Verifikation erfolgreich	offen
D4	0x39 parallel beobachten	keine trailing-byte- Warnung durch nachfolgenden inneren Frame	offen
D5	Passthrough während Download trennen	Download wird definiert abgebrochen; nach Reconnect sauberer Neustart möglich	offen
D6	App-Start	kein ERROR „Camera command not sent: transport is not initialized“	offen
D7	Foto-/Video-State	bestehende Capture- Diagnose unverändert; bekannte Busy/Mode- Probleme separat bewerten	offen

7. Offene Punkte

Die Ursache der wiederkehrenden WebSocket/Passthrough-Disconnects ist mit dieser Änderung nicht behoben; nur das Verhalten beim Disconnect bleibt kontrolliert.

Der Foto-Capture-Pfad zeigt weiterhin Device-busy-/Ready-State-Probleme und wird getrennt von diesem Download-Framing-Fix behandelt.

Ein realer 0.959-Gerätetest ist notwendig, bevor D2-D6 als bestanden markiert werden.

Da der Vite-Build in der aktuellen Linux-Umgebung blockiert ist, muss der Ziel-Build nach Erstellung prüfen, dass die 0.959-Webassets tatsächlich in app/src/main/assets/web erzeugt wurden.

8. Geänderte Dateien

app/build.gradle.kts

webui/package.json

webui/package-lock.json

webui/src/services/CameraMediaService.ts

README.md

PROTOCOL.md

VERSIONING.md

COMMIT_MESSAGE_CAMERA_RX_DOWNLOAD_FRAMING_2026-10-03_v0.959.txt

PATCH_CAMERA_RX_DOWNLOAD_FRAMING_2026-10-03_v0.959.patch

NUR_AENDERUNGEN_IN_ORIGINALSTRUKTUR_2026-10-03_CAMERA_RX_DOWNLOAD_FRAMING_v0.959.md

DOCUMENTATION/2026-10-03 - PotensicProxy_TAF - Projektdokumentation - Kamera-RX-Download-Framing - v1.6.docx

DOCUMENTATION/2026-10-03 - PotensicProxy_TAF - Projektdokumentation - Kamera-RX-Download-Framing - v1.6.pdf

9. Änderungshistorie

Version	Datum	Bearbeiter	Status	Änderung
v1.5	2026-10-02	OpenAI / ChatGPT	Arbeitsstand	0x39 CameraLogData strukturiert und Linux XOR-0x55 dekodiert.
v1.6	2026-10-03	OpenAI / ChatGPT	Implementiert / Test offen	Inneres FE-0x05/FF-FE- Reassembly, 0x1B- Framing-Härtung, Startup-Deferral, Software 0.959.